PAVAN DASAM

AI Engineer | Generative AI | Agentic AI | RAG

Over 3.5 Years • Financial Insurance | Health Insurance

✉ pavanwork2113@gmail.com

☎ +91 9640559197

◆ LangChain, LangGraph & MCP Certified

PROFESSIONAL SUMMARY

AI Engineer with over 3.5 years of experience designing, building and deploying enterprise Generative AI and Agentic AI solutions across Financial Insurance and Health Insurance domains. Hands-on experience developing production-oriented RAG applications, LLM-powered assistants and multi-step agentic workflows using Python, LangChain and LangGraph. Skilled across the full GenAI application lifecycle — prompt engineering, embeddings, vector search, retrieval pipelines, tool integration via the Model Context Protocol (MCP), structured outputs, evaluation and guardrails — with practical exposure to FastAPI-based API development and Docker/Azure deployment. Experienced collaborating with architects, product owners, SMEs and compliance stakeholders to translate regulated, enterprise business requirements into reliable, grounded and auditable AI systems.

TECHNICAL SKILLS

Programming	Python, Java, SQL
Generative AI	Generative AI, Large Language Models (LLMs), Prompt Engineering, RAG, Embeddings, Semantic Search, Context Management, Structured Outputs
Agentic AI	AI Agents, Tool Calling, Model Context Protocol (MCP), Agentic Workflows, State Management, Human-in-the-Loop, Multi-Agent Concepts
Frameworks	LangChain, LangGraph, FastAPI, Pydantic
LLMs	OpenAI GPT Models, Azure OpenAI, Anthropic Claude
Vector Databases / Search	FAISS, ChromaDB, Azure AI Search
RAG Pipeline	Document Loading, Chunking (RecursiveCharacterTextSplitter), Embeddings, Vector Search, Top-K Retrieval, Context Augmentation, Reranking Concepts, Metadata Filtering
AI Evaluation	RAGAS, LLM-as-a-Judge, Groundedness, Faithfulness, Relevance, Hallucination Evaluation
Responsible AI / Guardrails	Prompt Injection Protection, Content Filtering, PII Handling, Output Validation, Human Escalation
APIs / Backend	REST APIs, FastAPI, JSON, API Integration
Cloud / Deployment	Azure, Azure OpenAI, Docker, Git, GitHub, CI/CD Fundamentals

PROFESSIONAL EXPERIENCE

Agentic AI Engineer | Accenture

Client: Voya Financial – Financial Insurance • Generative AI & Agentic AI • Jun 2024 – Present

AI PRODUCTS

- **Autonomous Claims Processing Agent — Multi-Agent System**

Multi-Agent System — Coordinated Specialist Agents

Context: Voya's claims operations span supplemental health, life and disability, and in 2026 brought Leave/PFML/Short-Term Disability claims administration in-house for new business. Delivered a multi-agent system that coordinates several specialized

agents under a single LangGraph workflow to process a claim end-to-end.

```
Multi-Agent Workflow: New Claim → Intake Agent → Document Verification Agent → Policy Retrieval Agent → Eligibility Agent → Claims Analysis Agent → Fraud/Risk Agent → Recommendation Agent → Human Claims Examiner
```

- Designed and built a multi-agent system in LangGraph spanning Intake, Document Verification, Policy Retrieval, Eligibility, Claims Analysis, Fraud/Risk and Recommendation agents to process disability, life and supplemental health claims end-to-end.
- Developed the Intake Agent to classify incoming claims and extract claimant and policy information, and the Document Verification Agent to check received documents against claim-type requirements and flag missing items.
- Implemented the Policy Retrieval Agent using RAG over certificate/policy language to surface the applicable coverage sections for each claim.
- Developed the Eligibility Agent to validate coverage dates, plan eligibility and applicable rules via internal APIs and rules-engine integration, and standardized tool/data-source access across all agents using the Model Context Protocol (MCP).
- Built the Fraud/Risk Agent to flag unusual claim patterns for review, and the Recommendation Agent to assemble supporting evidence, cite policy sections, and produce a confidence-scored recommended next action for the human examiner.
- Implemented human-in-the-loop routing so examiners review and finalize every agent-recommended action before it is applied — e.g., a sample output flags 4/5 documents received, cites Policy Section 7.2, and recommends requesting the missing document at 94% confidence. Selected LangGraph over a simple LangChain chain for its persistent state, conditional branching and human-in-the-loop checkpoints.

Technologies: LangGraph, MCP, Azure OpenAI, Python, FastAPI, Azure Service Bus / Event Grid, Azure AI Search, PostgreSQL, Redis, Docker, AKS

• Voya Policy Servicing Agent — Single Agentic System

Single Agentic System — per Voya Financial's requirement

Context: Beyond claims, Voya's policy servicing team handles endorsements, beneficiary updates, billing inquiries and policy document requests. Rather than a multi-agent pipeline, delivered a single agentic system — one LLM-driven agent equipped with multiple tools — that reasons over each request end-to-end and decides which tool to call, matched to how self-contained this workflow is.

```
Single-Agent + Tools: Incoming Request → Single Agent (reasoning loop) → Tool Calls: Policy Change Tool | Billing Lookup Tool | Document Retrieval Tool | Compliance Check Tool → Response
```

- Designed a single agentic system — one LangGraph agent running a continuous reasoning-and-tool-calling loop, not a multi-agent pipeline — since policy servicing requests are typically self-contained and well-suited to one agent with the right tools.
- Exposed policy-change, billing-lookup, RAG-based document-retrieval and compliance-check capabilities as MCP tools that the single agent calls directly, keeping one continuous context and one audit trail per request instead of handing off between agents.
- Implemented persistent agent state so the same agent handles multi-turn requests — e.g., a beneficiary update followed by a compliance re-check — without losing context, with human-in-the-loop approval inside the same loop for changes above a defined risk threshold.
- Deliberately chose this single-agent, tool-calling design over a multi-agent pipeline for policy servicing, reserving the multi-agent approach for the higher-complexity claims workflow above.

Technologies: LangGraph (single-agent loop), MCP, Azure OpenAI, Python, FastAPI, Azure AI Search, PostgreSQL, Docker, AKS

• Voya Claims Intelligence Copilot — GenAI Copilot

Context: Voya's Claims Center supports claim initiation, document upload and status tracking. Examiners needed a faster way to understand large claim files spanning medical documents, claim forms, employer information, policy documents and correspondence.

```
RAG Architecture: Documents → OCR/Document Extraction → Chunking → Embeddings → Vector DB → RAG → LLM → Structured Claims Summary
```

- Built a GenAI copilot that ingests claim documents and produces a structured summary for examiners — claim type, reason, missing documents, relevant policy sections, potential issues and a recommended next step — instead of requiring manual review of the full file.
- Implemented OCR-based document extraction, chunking and embedding generation across heterogeneous claim file formats, and developed RAG retrieval over policy and claims documentation to ground each generated summary.
- Built FastAPI endpoints and a React-based interface for examiners to review AI-generated summaries alongside source documents, with Application Insights monitoring.
- Applied guardrails ensuring the copilot assists examiners with information synthesis only, and does not autonomously approve or deny high-impact insurance claims.

Technologies: Azure OpenAI / GPT, LangChain, Python, FastAPI, Azure AI Search, Blob Storage, PostgreSQL, React, Application Insights

Software Engineer | TAO Digitals

Client: Anthem – Health Insurance • Java Development & Generative AI • Mar 2023 – May 2024

GENAI ENGINEER — AI PRODUCTS

- **MemberAssist AI — GenAI Member Service Copilot**

Context: Members contact support with questions such as whether a procedure is covered, what their deductible is, why a claim was denied, expected out-of-pocket cost, provider network status and what an EOB means. Support agents needed to search across benefit documents, policy information, claims data and knowledge bases to answer them.

```
Architecture: Member/CSR → AI Assistant → Intent Detection → RAG → Member/Claims APIs + Vector DB → LLM → Guardrails → Response
```

- Built MemberAssist AI, a RAG-based GenAI assistant that retrieves a member's relevant plan, benefit and claims information and explains it in plain language for support agents and members.
- Implemented intent detection to route member questions — coverage, deductible, claim denial reason, cost-share, network status, EOB explanation — to the appropriate retrieval and API path.
- Developed RAG retrieval over benefit documents, policy information and internal knowledge bases, combined with real-time lookups against member and claims APIs.
- Example: for a denied-claim query, the assistant retrieves the claim, denial reason and policy, then explains the denial (e.g., missing prior authorization) with clear next steps such as contacting the provider or reviewing appeal options.
- Generated embeddings and indexed benefit guides and policy documents in a vector database, and built FastAPI endpoints with API gateway integration to expose the assistant to CSR-facing tools.
- Applied guardrails and grounding checks to keep responses accurate to member-specific plan data, and used LangSmith and Application Insights to trace, debug and monitor the assistant in production.

Technologies: Azure OpenAI / OpenAI, LangChain, Python, FastAPI, Azure AI Search, Vector Database, API Gateway, Azure Functions / AKS, Application Insights, LangSmith

JAVA DEVELOPMENT

- **Claims & Member Services Backend (Java)**

Context: Began as a Java backend developer on Anthem's claims and member services platform, building and maintaining the core services that GenAI applications would later integrate with, before transitioning into the GenAI engineering team.

- Developed and maintained backend microservices using Java, Spring Boot and REST APIs for claims processing and member service workflows.
- Designed and optimized SQL queries and stored procedures for claims and member data, and wrote unit/integration tests using JUnit and Mockito to ensure service reliability.
- Collaborated with QA and business teams to troubleshoot production issues and support release cycles via Git/Jenkins CI/CD in an Agile environment, then transitioned into the GenAI engineering team, applying this backend and API expertise to LLM-powered application development.

Technologies: Java, Spring Boot, REST APIs, SQL, JUnit, Mockito, Git, Jenkins, Agile

EDUCATION

Bachelor of Technology (B.Tech)

JNTUK University • 2023

CERTIFICATIONS

- Certified in Model Context Protocol (MCP) – Anthropic
- Databricks Certified – Generative AI
- LangChain Certification
- LangGraph Certification